

Filter Banks, Texture & Learned Features

CS 4243 Computer Vision & Pattern Recognition

Angela Yao

Recap & Outline

Last Week

- Motivation for studying image edges
- Derivative filters to extract gradients
- Need for smoothing
- Canny edge detector
- Line detection with Hough transform

Today's Lecture

- Gabor Filters
- Filter banks
- Multiscale reasoning
- Textures
- Precursor to CNNs

The Big Picture

We have looked at so far

- How digital images are represented
- Point-wise and filtering operations to adjust the appearance
- Extracting low-level structures (boundaries, regions)
 - Boundaries: image gradients, edges, lines
 - What about regions?
 - Characterized by local patterns or textures.
 - At what scale should we examine the patterns?
 - Can we learn filters from the data direction?

L4 today

Lecture 1: 1- or 3-channel
matrices of intensity values

Lecture 2: brightening,
contrast, equalization, etc.

Lecture 3: Derivative of Gaussian
filter, Canny, Hough Transform

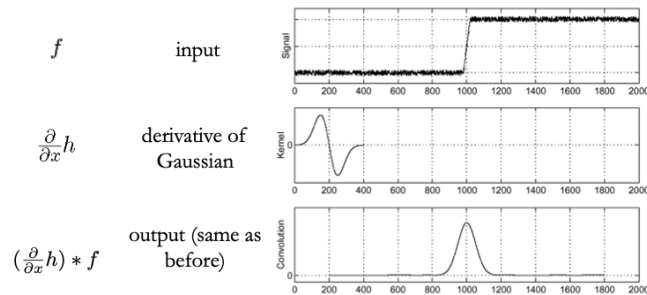
Filter Banks

Gabor filters, filter bank responses as a representation

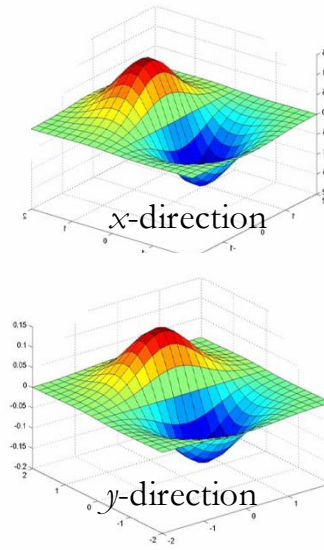
Recall the Derivative of Gaussian Filter

Derivative of Gaussian Filter

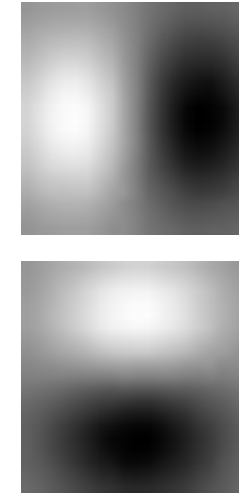
Differentiation Property of Convolution: $\frac{\partial}{\partial x}(h * f) = (\frac{\partial}{\partial x}h) * f$



1D

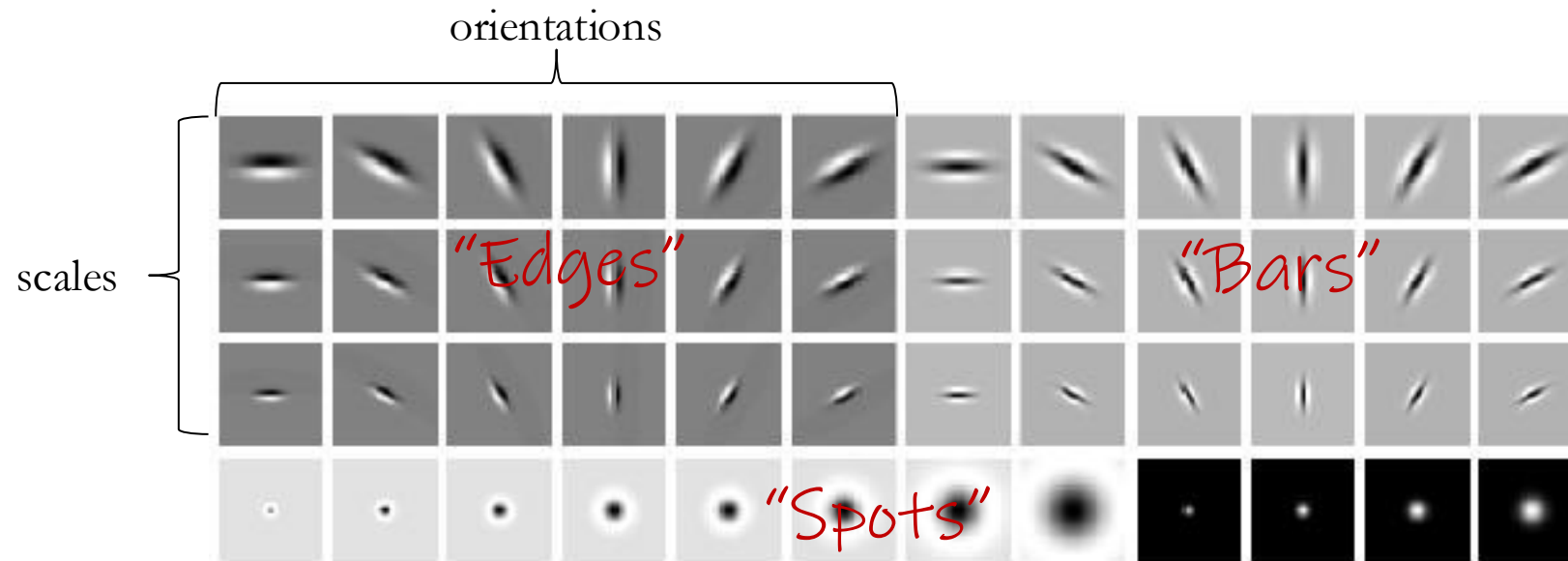


2D



- Derivative of Gaussian filter: smoothing + derivative operation to extract image gradients
- 2D Gaussian can be arranged in different ways
 - Different σ ; aligned to x-/y-axes or even rotate in 2D space

Filter Banks are a Collection of Multiple Filter Kernels

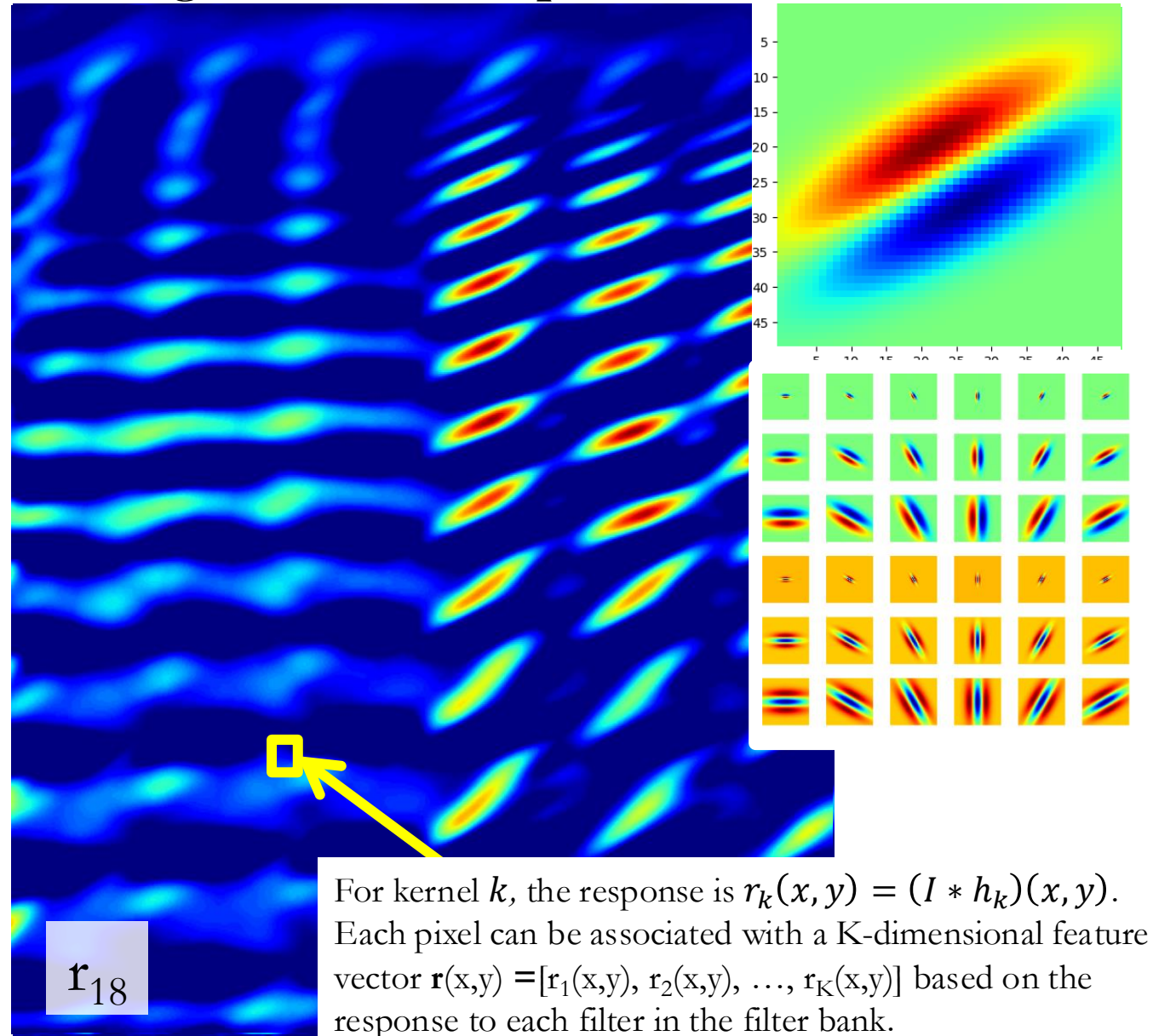


- Each filter extracts a particular local pattern
- Several filters will provide coverage over a combination of local patterns

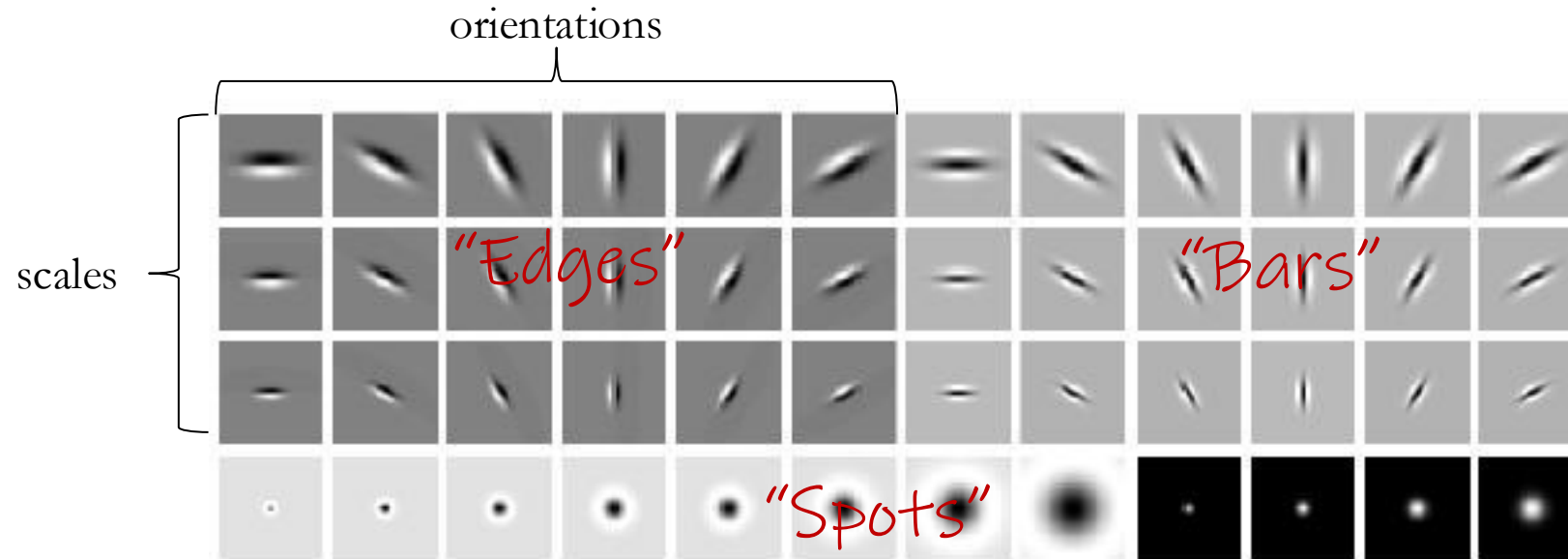
Magnitude of Response



Esplanade's roof cladding



What Filters to Put in the Filter Bank?

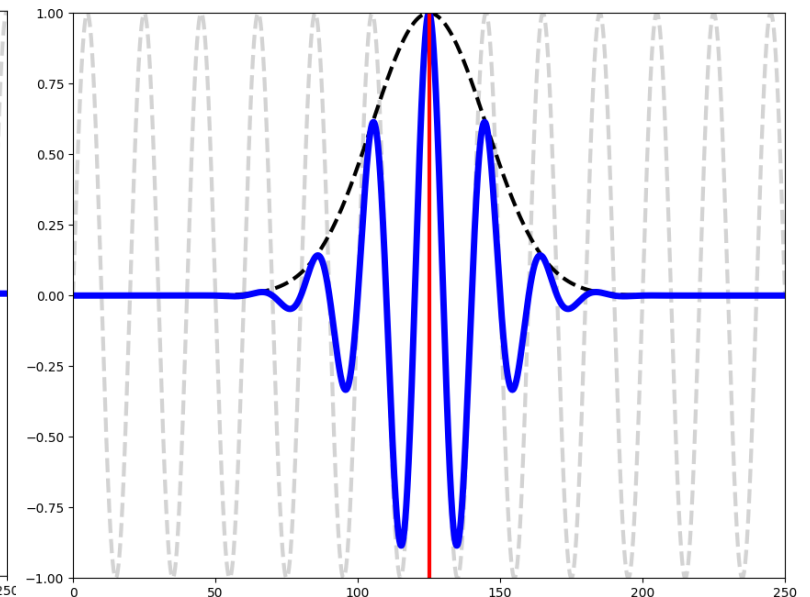
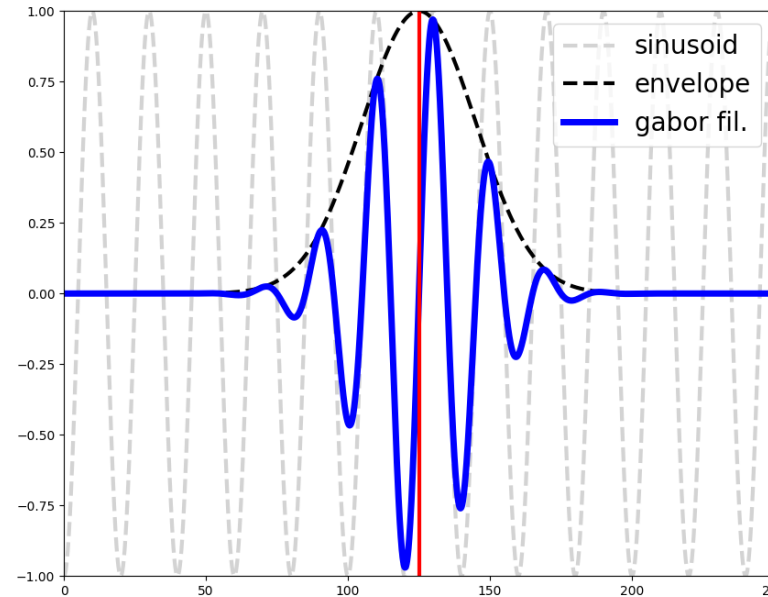


- A combination of different patterns at various scales & orientations
 - We can select them ourselves manually or ...
 - Fix a general mathematical function with a range of parameters, e.g. [Gabor Filters](#).

Gabor Filters (in 1D)

- Combining sinusoids with an exponential (Gaussian) envelope.
- Two important parameters:
 - f : frequency selects spacing between the sinusoidal waves
 - σ : scale determines the size of the region covered by the Gaussian envelope

Gaussian envelope
shapes the sinusoid



$$e^{-\frac{x^2}{2\sigma^2}} \sin(2\pi f x)$$

scale frequency
of sinusoid

exponential envelope sinusoid term

$$e^{-\frac{x^2}{2\sigma^2}} \cos(2\pi f x)$$

* f corresponds to $\frac{1}{\lambda}$ in next slide



Gabor filters

2D Gabor Filters

$$f_{mn} = \underbrace{\frac{1}{2\pi\sigma^2} \exp\left[-\frac{m^2 + \gamma^2 n^2}{2\sigma^2}\right]}_{\text{exponential (Gaussian) envelope}} \underbrace{\sin\left[\frac{2\pi(\cos[\omega]m + \sin[\omega]n)}{\lambda} + \phi\right]}_{\text{2D-Sinusoid**}}$$

exponential (Gaussian) envelope
selects a local area

2D-Sinusoid**
selects orientation and frequency

m, n – 2D coordinates of the kernel

ω – orientation of sinusoid in 2D, influences the orientation of detected feature

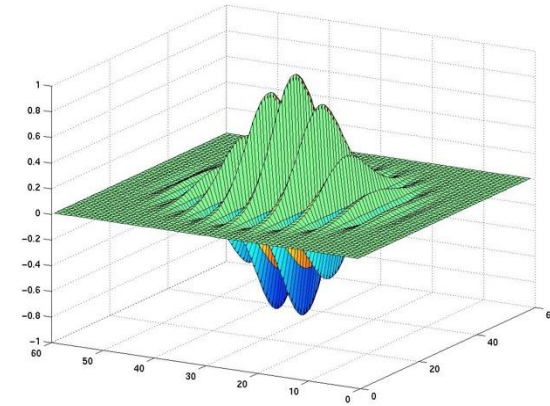
λ – wavelength of sinusoid (inversely proportional to the frequency)

ϕ – phase of sinusoid; shift from center of kernel

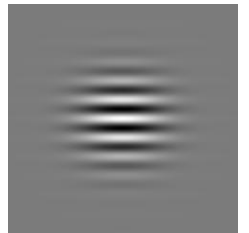
γ – aspect ratio of envelope coverage ($\gamma = 1$ means circular, otherwise elliptical)

σ – rate of decay of exponential envelope

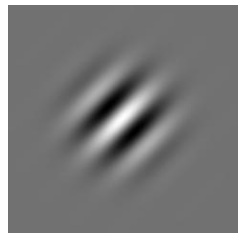
** (we usually set λ proportional to σ so a constant number of cycles is visible)



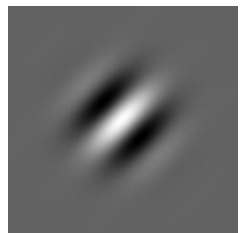
High frequency
along axis



Lower frequency
(diagonal)



Even lower
frequency





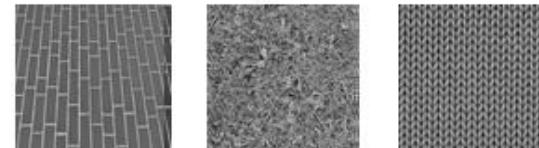
2D Gabor Filter Kernel Responses

Input Images

brick

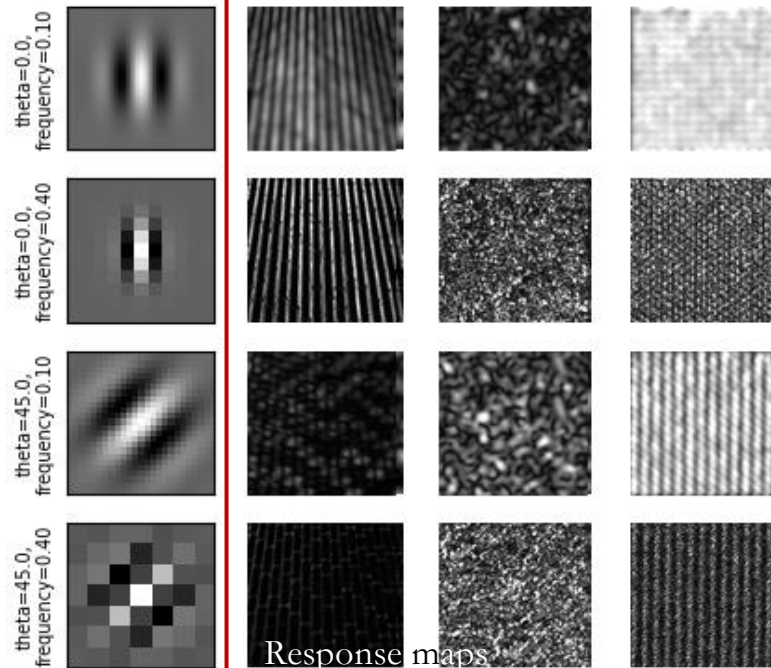
grass

knit



4 Filters w/ dif.
orientations &
frequencies

*** note they are
different sizes*



- **Orientation:** vertical (1,2) vs. diagonal (3,4) patterns.
- **Frequency:** lower frequency (1,3) have broader, smoother responses to coarser patterns; higher-frequency filters (2,4) with smaller, more localized responses
- **Image Structure:** brick and knit produce organized, directional responses; grass is irregular and distributed.
- **Frequency vs. Scale:**
Frequency controls the sinusoid oscillation spacing
Gaussian scale controls the spatial neighbourhood
Effects coupled, hence tying the two together

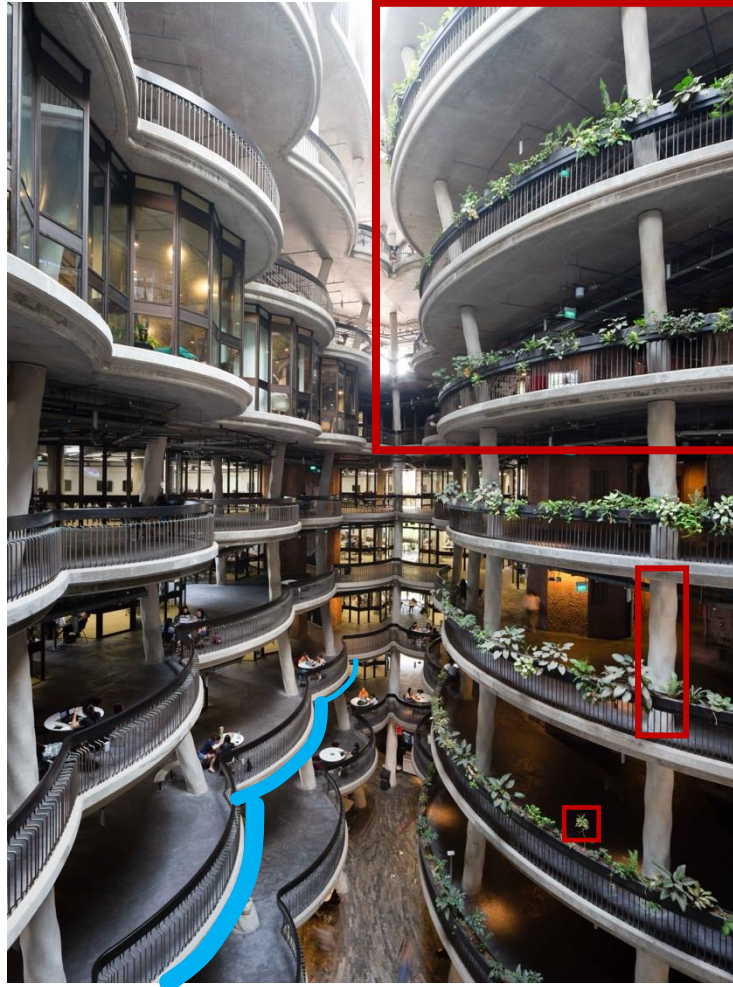
Multi-Scale Reasoning

Scale variations, image pyramids, scale selection & multi-scale representations

A Single Image Has Structures at Many Scales

The same **structure** can have different apparent sizes in the scene.

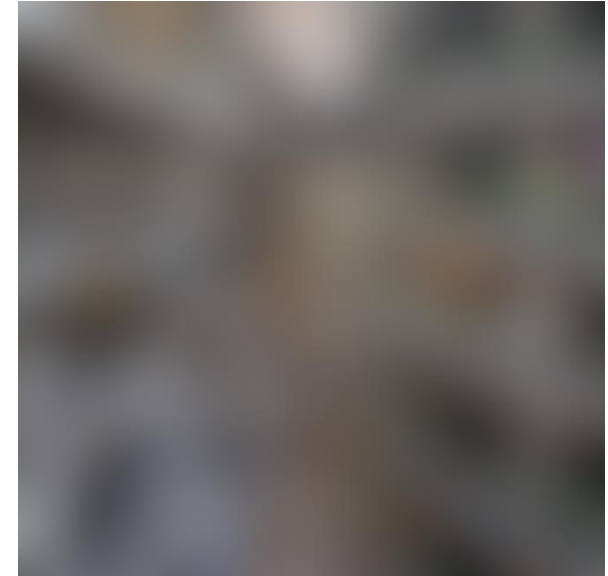
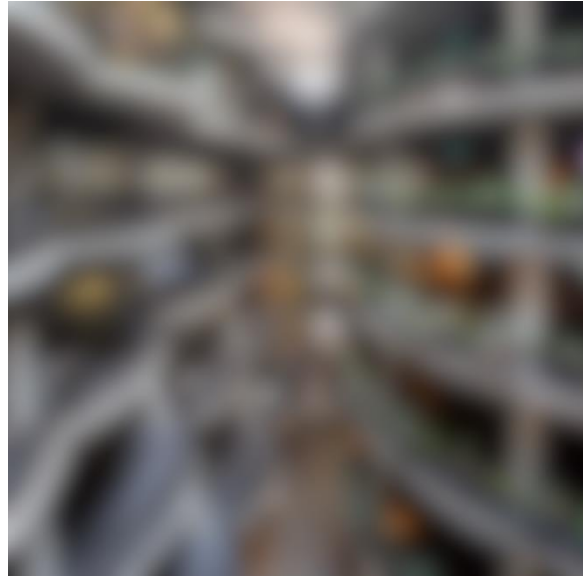
Differences occur due to distance to the camera.



Different **structures** have different characteristic sizes in a scene.

Compare the plant vs. the column vs. the building itself.

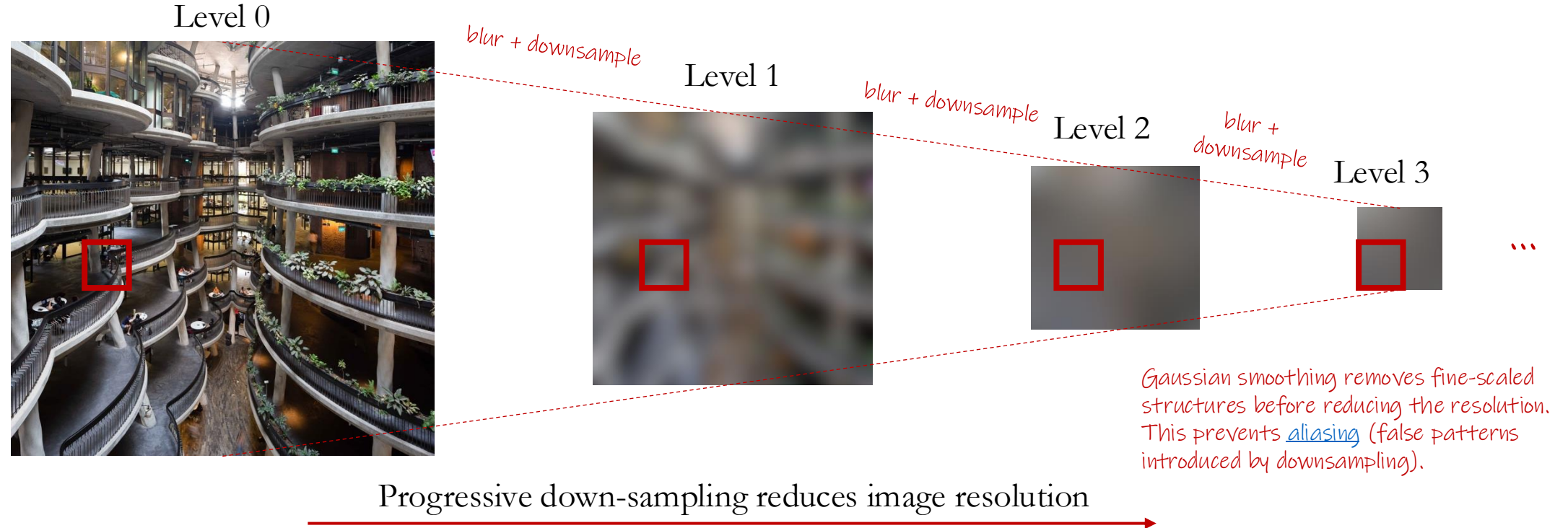
Smoothing Suppresses Details



Gaussian smoothing with progressively larger σ

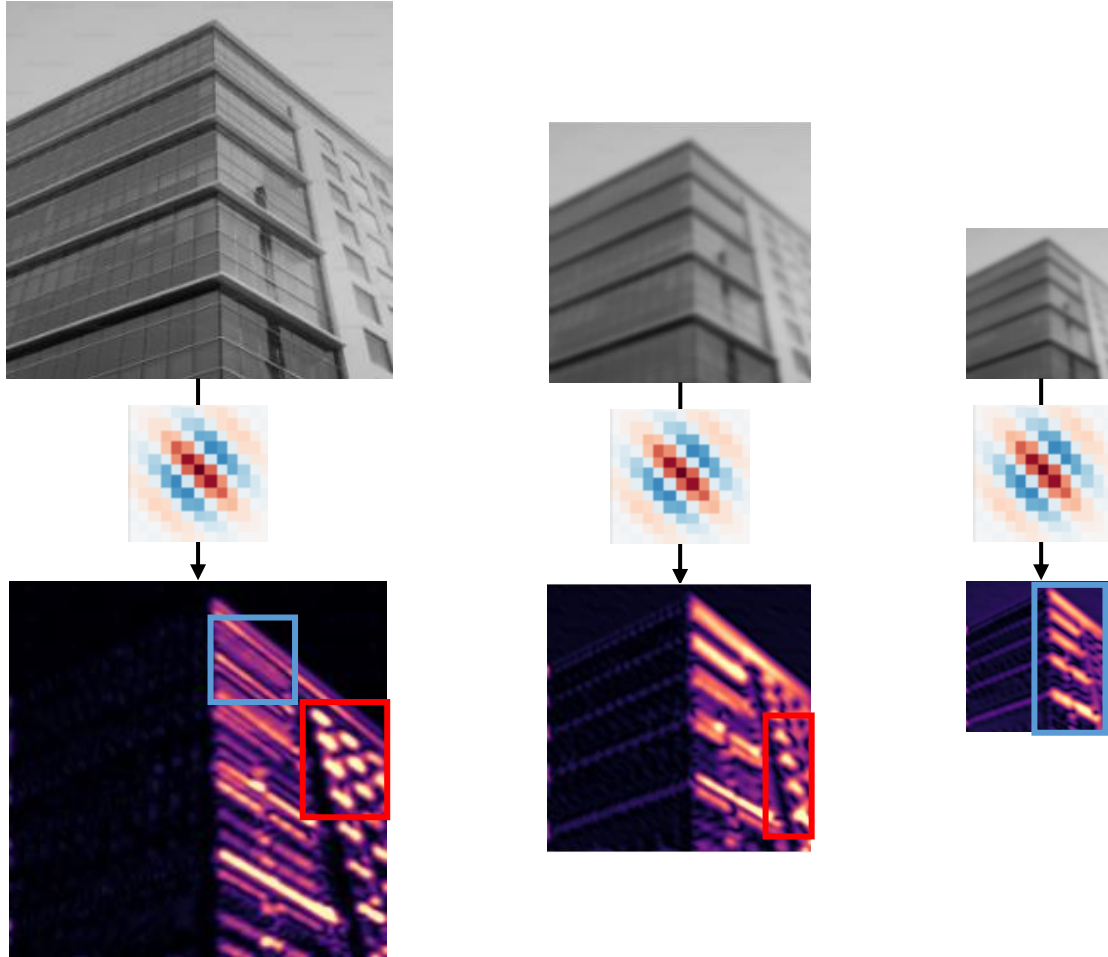
- Ignore the variations over smaller distances to focus on larger structures
- Smoothing removes the structures smaller than the scale we want to examine

Image Pyramids Represent Multi-Scale Information



- A Gaussian image pyramid is a collection of progressively blurred & downsampled images
- Downsampling: removing rows and columns of pixels at a fixed stride or downsampling rate
- Note how the same sized window covers different image scales in the pyramid

The Same Filter Sees Larger Regions At Higher Pyramid Levels



- The same filter responds to larger structures higher up in the pyramid
- Larger structures can come from physical or apparent differences
 - Window frames vs. building stories
 - “Bigger” windows towards bottom
- Achieve multi-scale coverage with one filter applied to different image scales *or* different scales of filters applied to one image

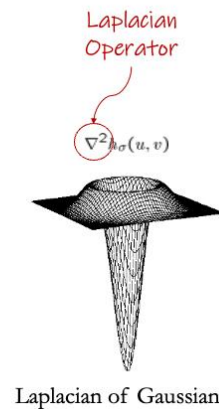
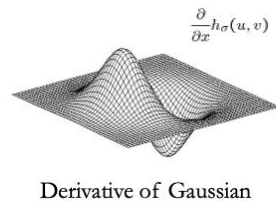
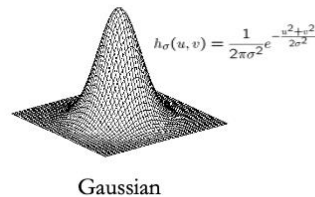


What Do We Do With Multi-Scale Responses?

(1) Scale Selection:

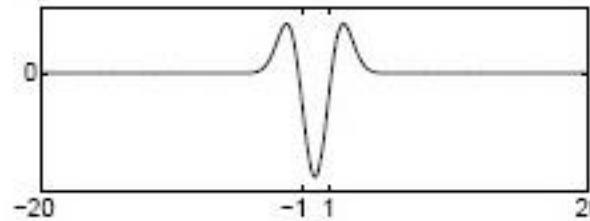
- Measure a response across multiple scales and choose a local maximum
- The response peaks when the filter scale matches the structure size
- Useful for detecting specifically-scaled structures in a scene

2D Gaussian filters

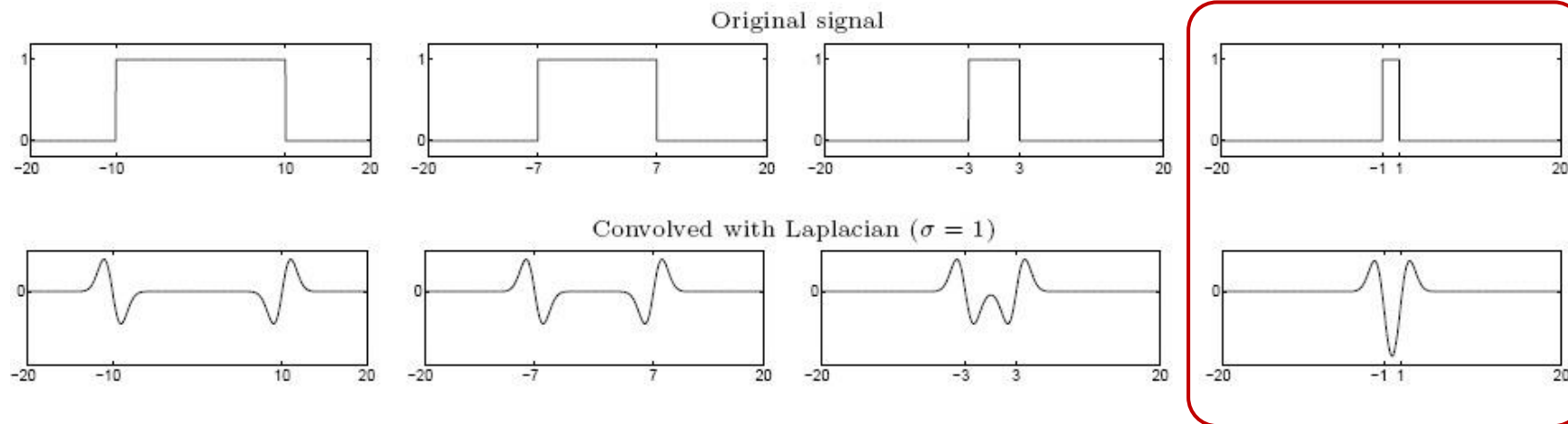


Recall the Laplacian of Gaussian
Filter
(second derivative of Gaussian)

Laplacian of Gaussian Filter



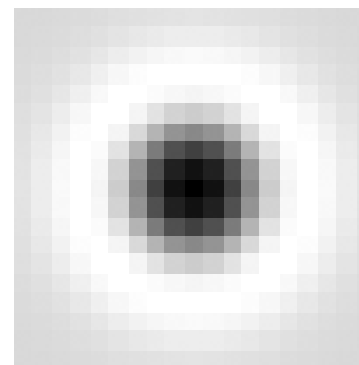
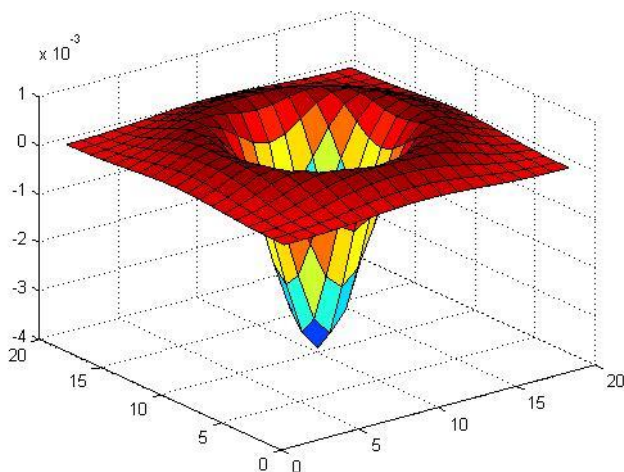
2nd derivative
of Gaussian



Highest response when the signal has the same
characteristic scale as the Gaussian.

Laplacian of Gaussian (LoG) – 2D

“Blob detector”

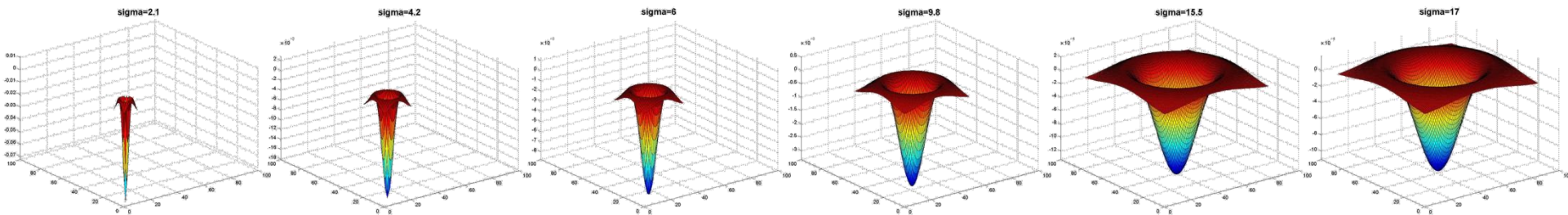


$$\nabla^2 g = \frac{\partial^2 g}{\partial x^2} + \frac{\partial^2 g}{\partial y^2}$$

Laplacian
operator

Very similar to a difference of Gaussians – i.e. a Gaussian minus a slightly smaller Gaussian (cheaper approximation than taking 2nd derivative)

Applying different Laplacian filters

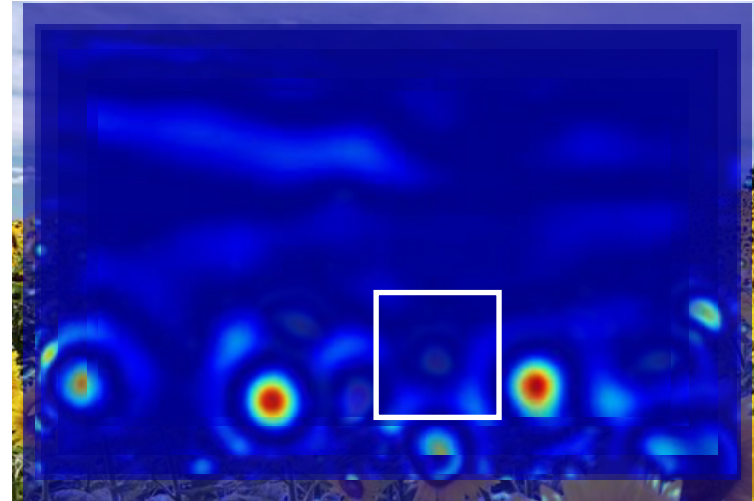
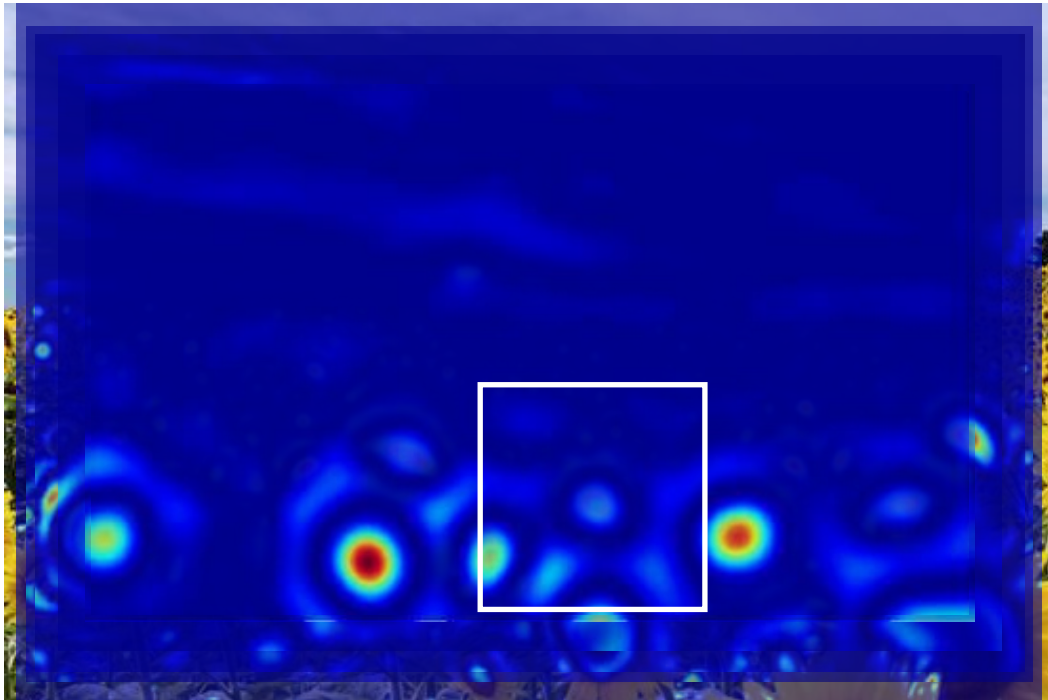
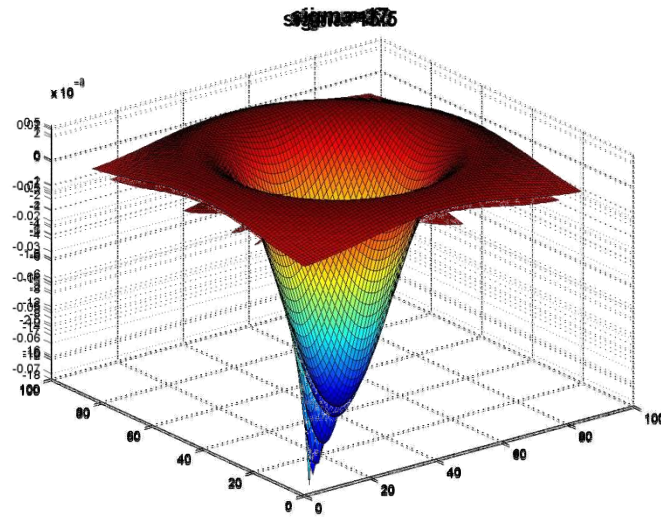


Full size

3/4 size

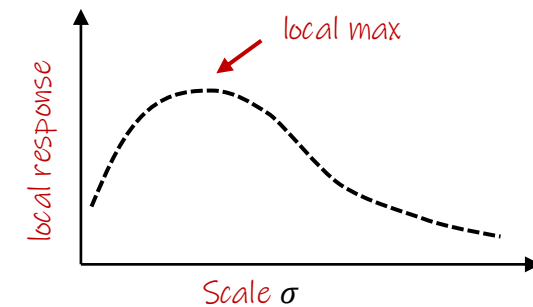


Focus on what happens to these 2 flowers.

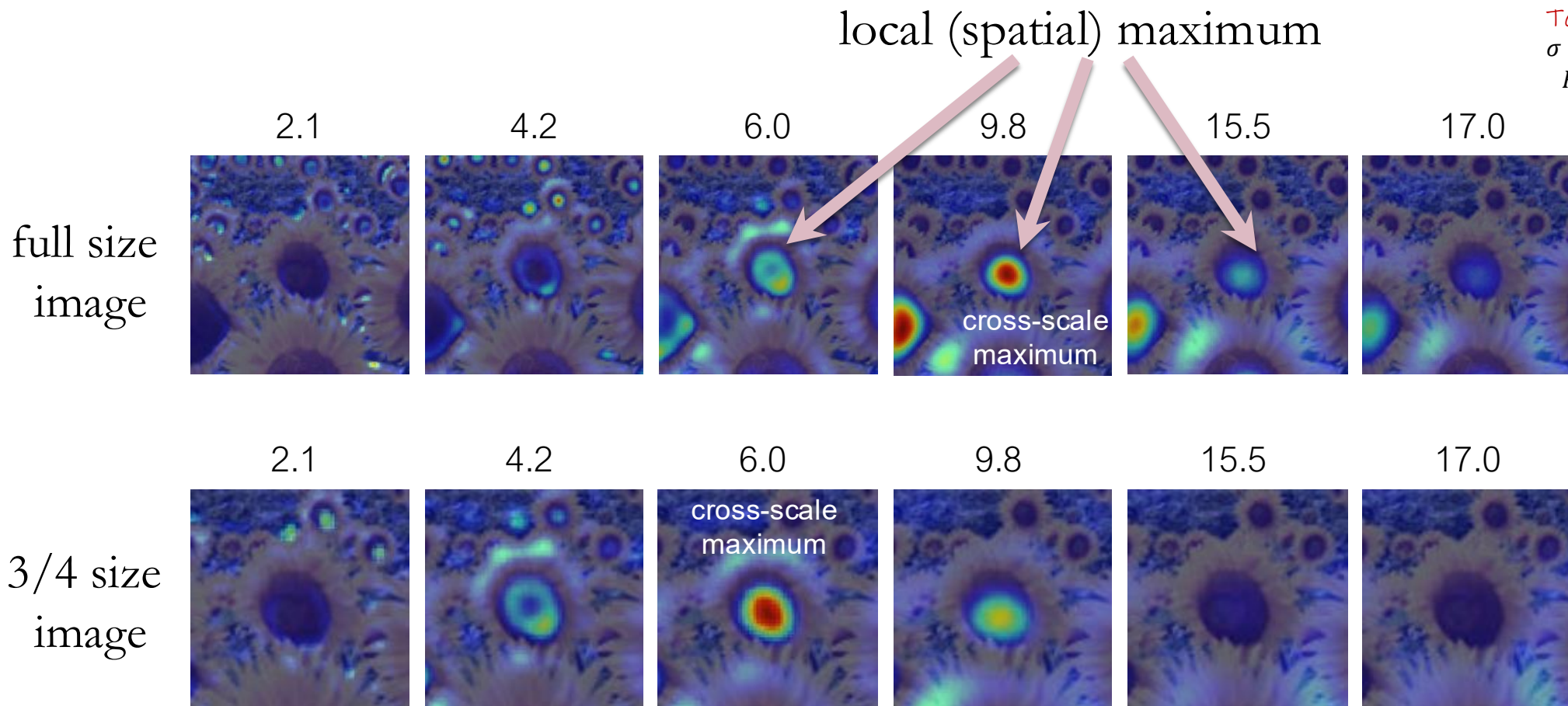


jet color scale
blue: low, red: high

Finding the Characteristic Scale



To compare responses at different σ directly, need to normalize:
 $R(x, y; \sigma) = \sigma^2 |\nabla^2 (G_\sigma * I)(x, y)|$



What Do We Do With Multi-Scale Responses?

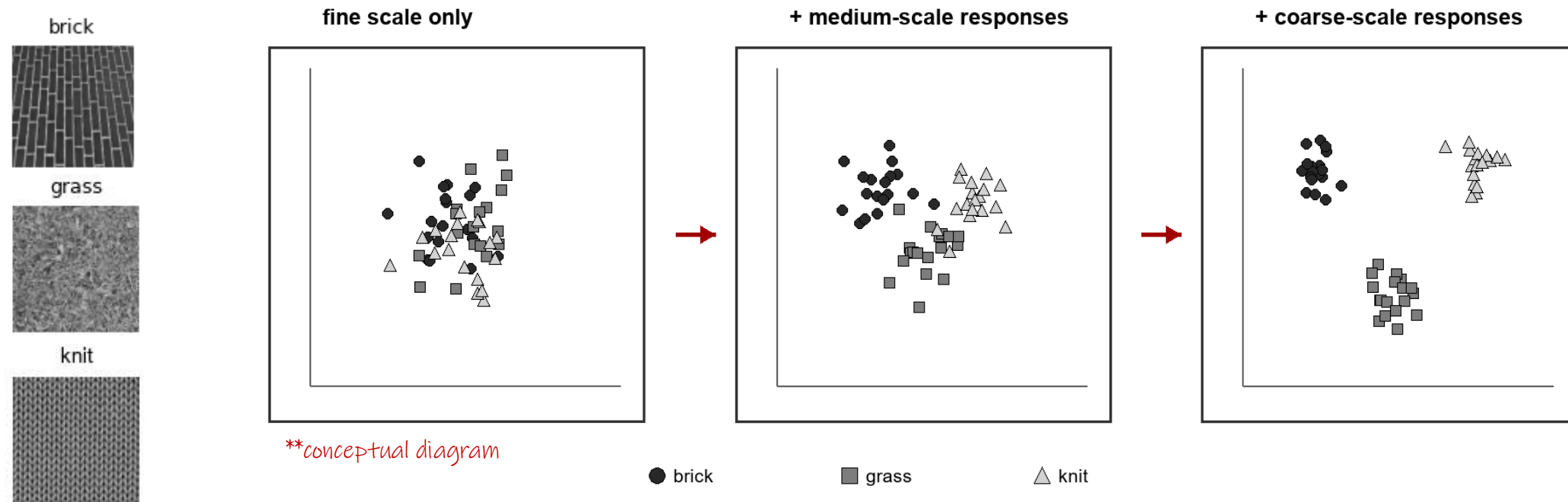
(1) Scale Selection:

- Measure a response across multiple scales and choose a local maximum
- The response peaks when the filter scale matches the structure size
- Useful for detecting specifically-scaled structures in a scene

(2) Multi-Scale Representations

- Keep responses from several scales and aggregate it as part of a feature vector:
 - Single-scale $\mathbf{r} = [r_1, r_2, \dots, r_K]$ vs. multi-scaled $\mathbf{r} = [r_1^{\sigma_1}, r_2^{\sigma_1}, \dots, r_K^{\sigma_1}, r_1^{\sigma_2}, r_2^{\sigma_2}, \dots, r_K^{\sigma_2}]$
- Useful when the scene contains structures of different scales, or when multiple scales of information are more discriminative

Multi-Scale Representations



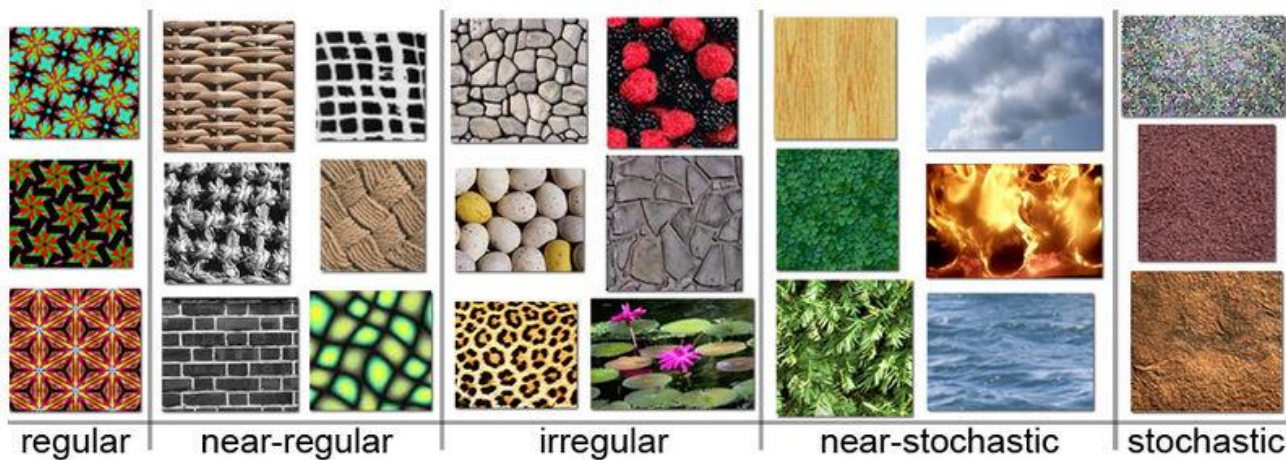
Information from multiple scales are complementary and may lead to better separation.

Textures

Visual textures, regional pooling, texture vectors, material classification

Image Textures

A pattern with repeating elements. Repetitions can have some variation, but can still separate what repeats vs. what stays the same



Visually, a texture must be perceived as a pattern instead of individual elements



VS



Why Analyze Visual Textures?

For visual perception:

- Indicative of material properties
- Important appearance cue (shapes, boundaries, textures), especially if shape is similar across objects

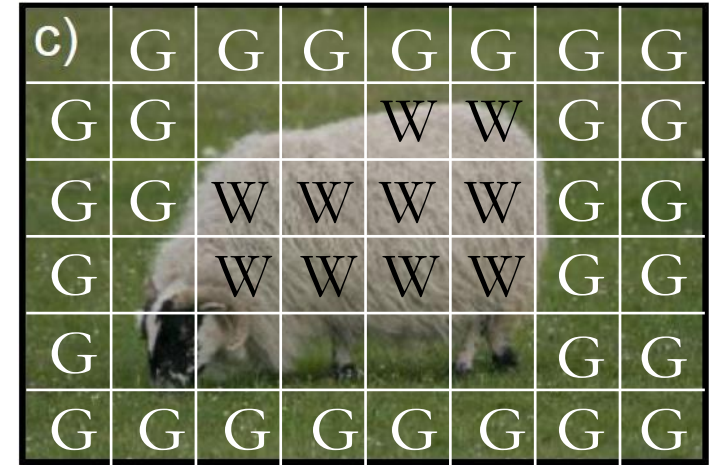
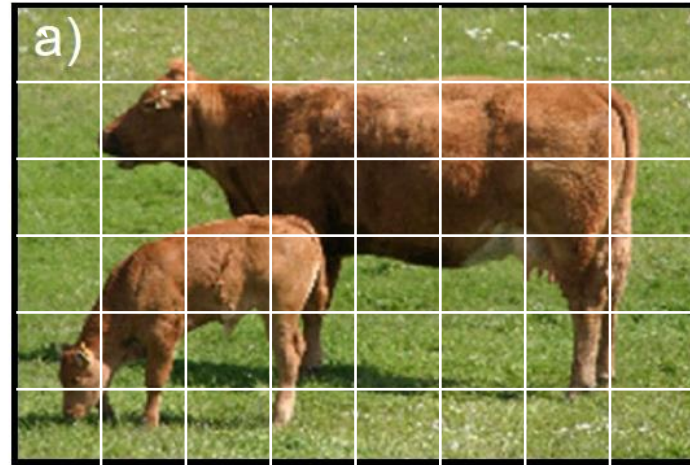
For computer vision:

- Representation-wise, we want a feature one step above basic building block of colour, simple filters, and or edges.

Mathematical Representations of Texture

Regions in the scene can be treated as textures:

- Grassy textures
- Woolly textures
- ...



How can we characterize these regions of different textures?

Apply a filter? A filter bank? A Multi-scale filter bank?

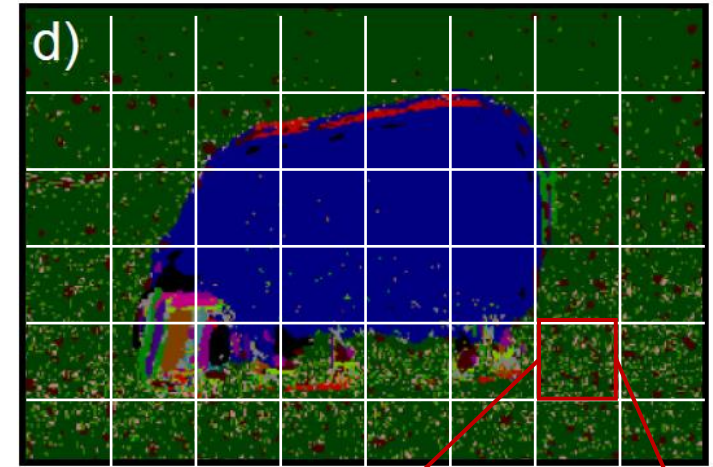
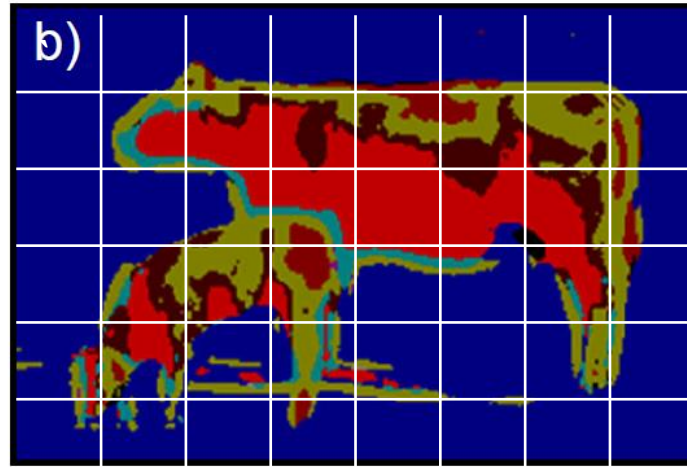
Consider our feature response $\mathbf{r}(x, y) = [r_1(x, y), r_2(x, y), \dots, \underline{r_K}(x, y)]$
at each pixel (x,y) by applying a filter bank.

*K filters of dif. orientations,
frequencies, scale, etc.*

Mathematical Representations of Texture

What if we clustered the feature responses?

Pixel-wise responses are too localized to describe the regional pattern that constitutes texture.



***Each colour denotes a feature cluster, based on a characteristic response to the filter bank.*

Pooling merges the local responses into a regional feature $\mathbf{t}$

$$\mathbf{r}(x, y) = [r_1(x, y), r_2(x, y), \dots, r_K(x, y)]^\top$$

Response of filter k at pixel (x,y)

$$t_k(\Omega) = \frac{1}{|\Omega|} \sum_{(x,y) \in \Omega} |r_k(x, y)|$$

Pooled response of filter k across region Ω

$$\mathbf{t}(\Omega) = [t_1(\Omega), t_2(\Omega), \dots, t_K(\Omega)]^\top$$

Pooled texture feature for region Ω





Comparing Feature Vectors

- Two features a and b can be compared with a distance function D
- A simple D is the Euclidean distance function

$$D(\mathbf{a}, \mathbf{b}) = \|\mathbf{a} - \mathbf{b}\| = \sqrt{(a_1 - b_1)^2 + (a_2 - b_2)^2}$$

- We can compare textures from two regions Ω_a and Ω_b

$$D(\Omega_a, \Omega_b) = \|\mathbf{t}(\Omega_a) - \mathbf{t}(\Omega_b)\|_2$$

- A small D indicates similar textures; a large D indicates dissimilar textures



Two Simple Material Classifiers



Nearest neighbour compares the texture vector of the query wrt all the training image patches; query takes the class of the patch with the smallest distance, i.e. the 'nearest' (in feature space)

Nearest class mean computes a mean texture vector for each class; compares the texture vector of the query wrt all the mean vectors; query takes the class of nearest mean vector

Suppose we are given M labelled training patches belonging to C material classes:

$$\{(\Omega_i, y_i)\}_{i=1}^M, \quad y_i \in \{1, \dots, C\}$$

For a query patch Ω :

(1) Nearest-Neighbour Classifier

$$i^* = \arg \min_i \|\mathbf{t}(\Omega) - \mathbf{t}(\Omega_i)\|_2, \quad \hat{c}(\Omega) = y_{i^*}$$

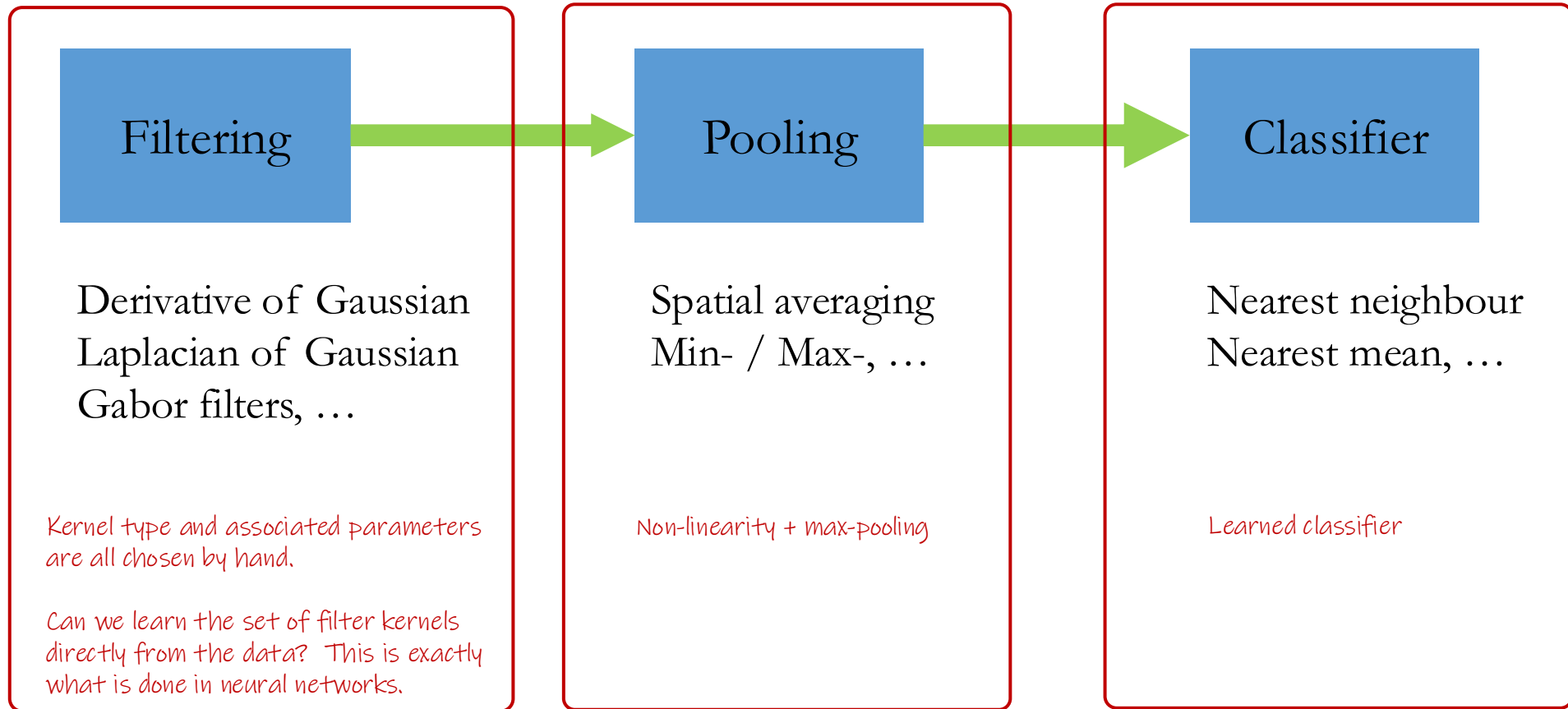
(2) Nearest class mean / representative texture vector

$$\mu_c = \frac{1}{N_c} \sum_{i: y_i=c} \mathbf{t}(\Omega_i), \quad \hat{c}(\Omega) = \arg \min_c \|\mathbf{t}(\Omega) - \mu_c\|_2$$

Learned Filters

Learned kernels, convolutional layers, ReLU, Max Pooling

From Hand-Designed to Learned Kernels



What Does it Mean to Learn a Filter Kernel?

- For input I , the feature response r_k , for a given kernel f_k is given by convolving the input with the kernel:

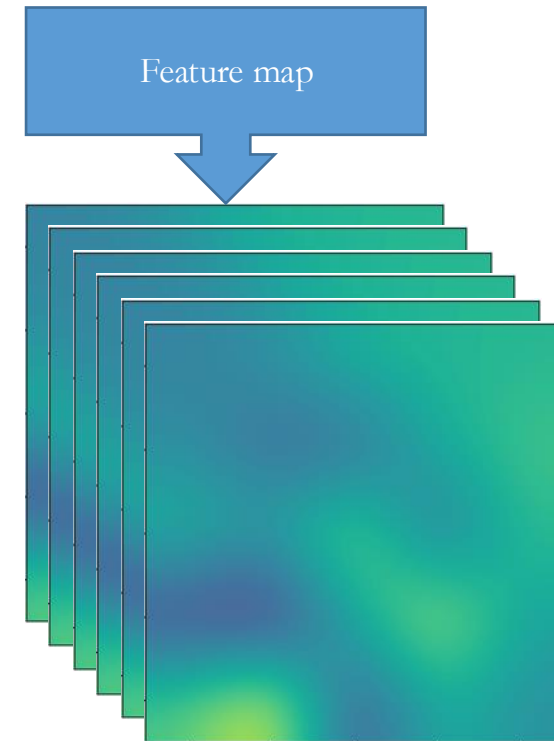
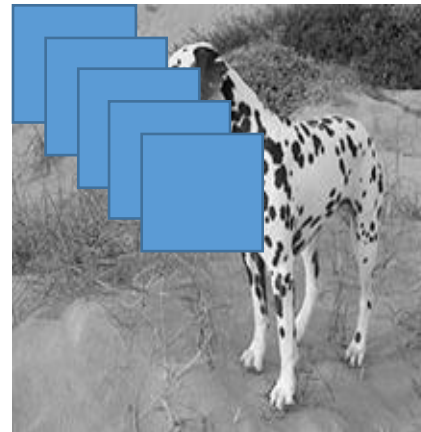
$$r_k = I * f_k$$

- A designed kernel f_k has weights set manually or by some fixed mathematical function
 - A learned kernel f_k has weights which are adjusted using training data, so as to lower the prediction error (the mechanism, backpropagation, is out of our scope)
 - Both designed and learned kernels are convolved with the input
- Convolutional neural networks learn K kernels at each convolutional layer
- We can also think of this as learning a filter bank at each layer

Convolution Layers Are Learned Filter Banks

- Convolving a filter on a single input gives a single feature map
- Convolving multiple filters (a filter bank!) gives a multi-channelled feature map
- Channels are stacked into multi-dimensional feature responses (becoming inputs for subsequent layers)

5.4	0.1	3.6
1.8	2.3	4.5
1.1	3.4	7.2



A Non-Linearity Turns a Filter Response into an Activation

- Rectified Linear Unit (ReLU) are the preferred non-linearity for most deep networks:

$$a = \text{ReLU}(r) = \max(0, r)$$

- Without nonlinearities, several linear convolution layers collapse into one linear operation.

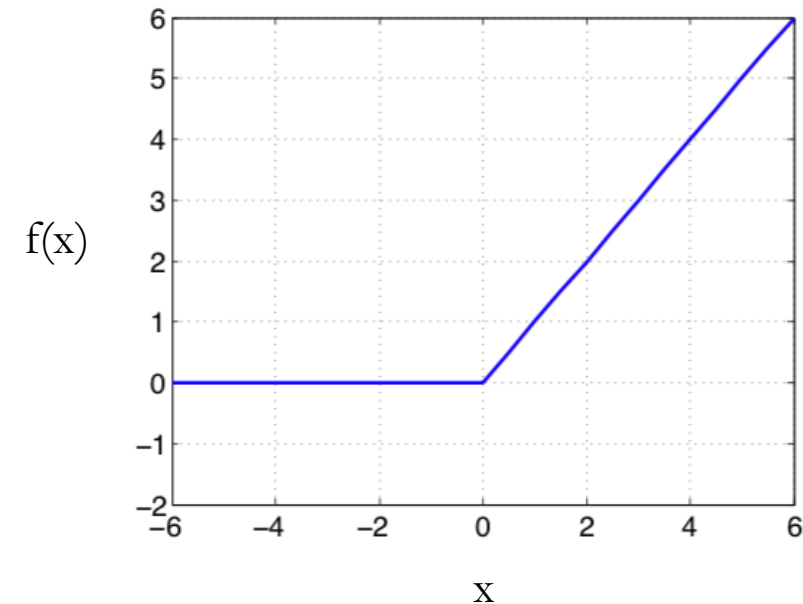


Figure: Rectified Linear Unit

Theoretical and practical advantages of ReLU (vs. other non-linearities) are beyond our scope.

The Necessity of Non-Linearities

$$\mathbf{x} \rightarrow \boxed{\mathbf{k}_1 * \overset{r_1}{\phantom{\mathbf{x}}}} \rightarrow \boxed{\mathbf{k}_2 * \phantom{\mathbf{x}}} \rightarrow \mathbf{r}_2$$

$$r_1 = \mathbf{k}_1 * \mathbf{x}$$

$$\begin{aligned} r_2 &= \mathbf{k}_2 * r_1 \\ &= \mathbf{k}_2 * (\mathbf{k}_1 * \mathbf{x}) \\ &= (\mathbf{k}_2 * \mathbf{k}_1) * \mathbf{x} \\ &= \mathbf{k}_{\text{eff}} * \mathbf{x} \end{aligned}$$

The associative property of the convolution operation allows $\mathbf{k}_1$ and $\mathbf{k}_2$ to be combined into a single effective kernel $\mathbf{k}_{\text{eff}}$.

$$\mathbf{x} \rightarrow \boxed{\mathbf{k}_1 * \overset{r_1}{\phantom{\mathbf{x}}}} \rightarrow \boxed{\text{ReLU}} \rightarrow \boxed{\mathbf{k}_2 * \phantom{\mathbf{x}}} \rightarrow \mathbf{r}_2$$

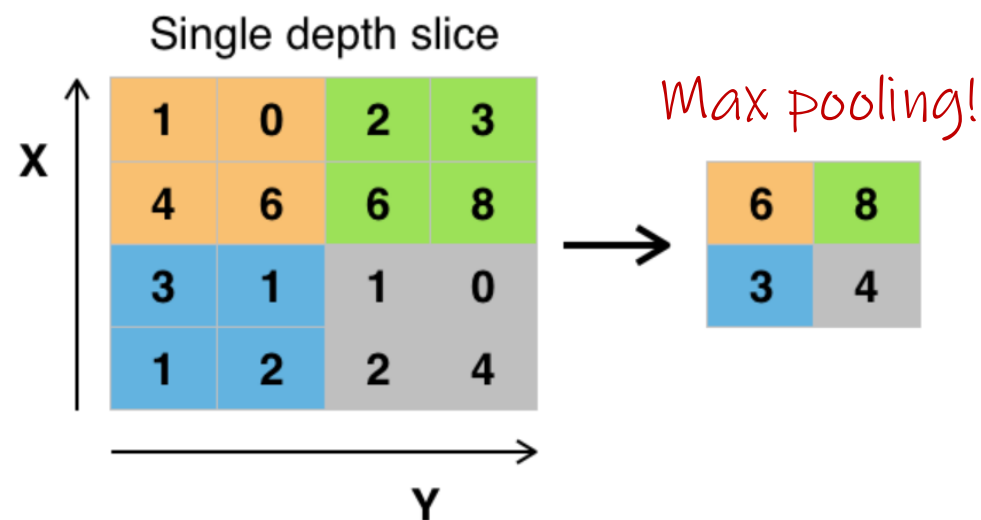
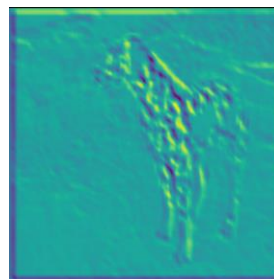
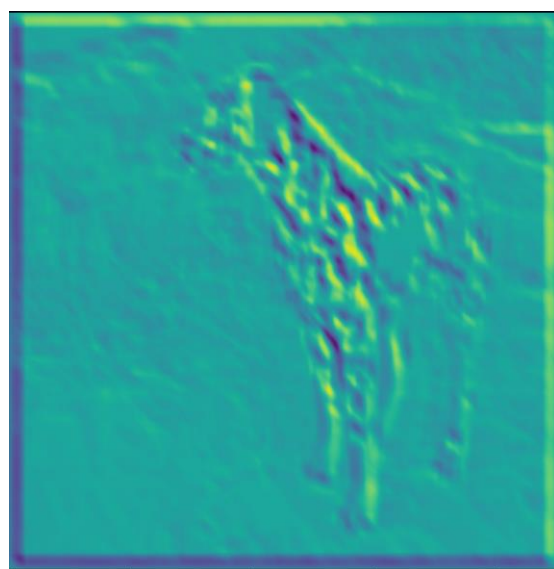
$$r_1 = \mathbf{k}_1 * \mathbf{x}$$

$$\mathbf{r}_2 = \mathbf{k}_2 * \text{ReLU}(\mathbf{k}_1 * \mathbf{x})$$

Because ReLU is a non-linear operation, so $\mathbf{k}_1$ and $\mathbf{k}_2$ can no longer be associated into one kernel.

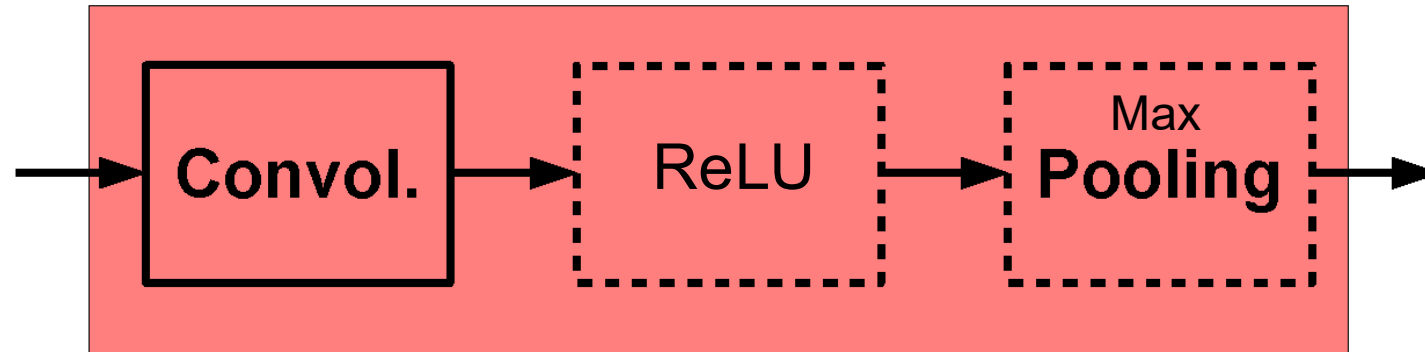
Max Pooling

Pooling makes large distortions become smaller, so less sensitive

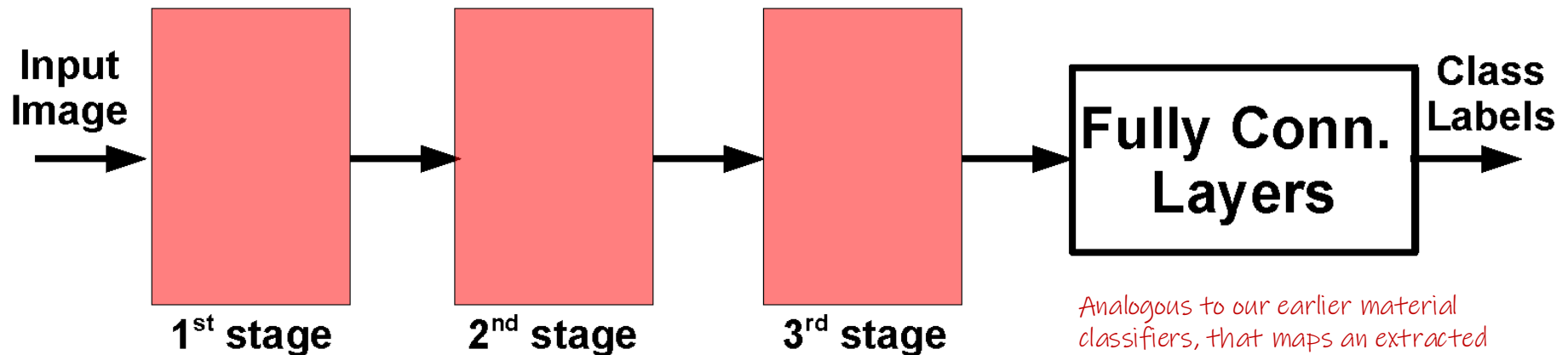


Putting it all together ...

One stage (zoom)



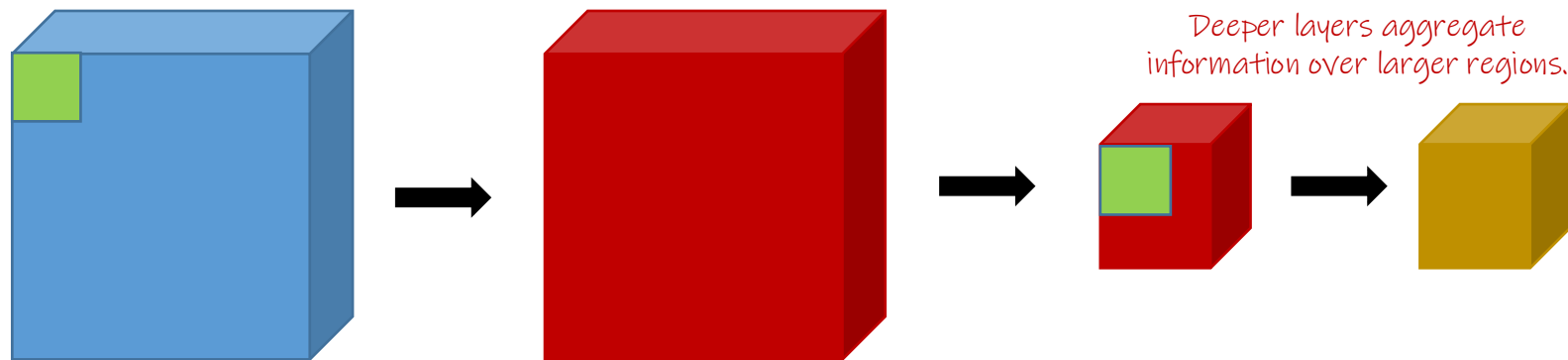
Whole system



Analogous to our earlier material classifiers, that maps an extracted feature vector to class scores.

Convolution-pooling-convolution

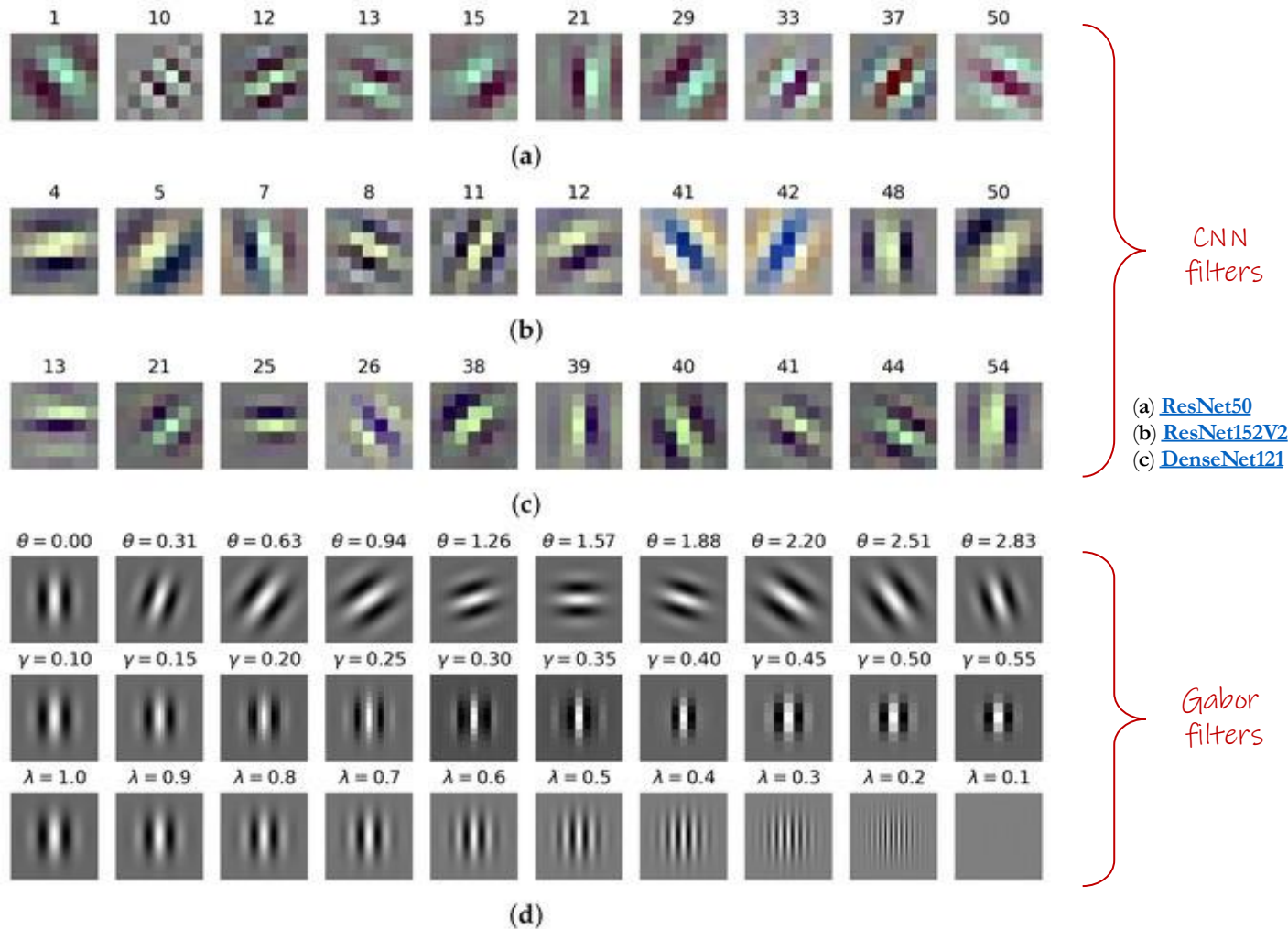
- Interleaving convolutions and pooling causes later convolutions to capture a larger fraction of the image with the same kernel size
- The receptive field is defined as the set of image pixels that an intermediate output pixel depends on
- Convolutions after pooling increase the receptive field



What Does This Look Like?



Similarities Between Learned vs. Gabor Kernels



- Early CNN layers often learn Gabor-like filters, sensitive to oriented light-dark transitions at different frequencies.
- These are broadly useful building blocks for processing natural images and are learned across many architectures and tasks.
- Deeper filters beyond the first or early layers operate on feature maps and are less visually interpretable.

A CNN Extends the Filter-Bank Pipeline

	Filter Banks	CNNs
Kernels	Manually specified	Learned from data
Depth	Shallow (usu. one filter bank)	Deep (many stacked convolutional blocks)
Spatial Aggregation	Pooled over a region to form one feature vector	Applied locally w/ stride to reduce resolution while preserving the approximate spatial layout.
Kernel + classifier	Classifier trained after	Learned jointly end-to-end

Summary

Filter Banks

- Apply several kernels of various types, orientations, scales, etc. to measure different local image patterns.
- Gabor filters allow for parametric selection of scale, orientation, and frequency

Texture Representation

- Pool local filter responses over a region to form a texture vector.
- Texture vectors can be compared or used for regional characterization and classification.

Multi-Scale Reasoning

- Smoothing suppresses fine detail; pyramids represent progressively coarser resolutions.
- Responses across scales can be selected or retained as a multiscale representation.

Learned Filters

- Convolutional layers have learned filter banks based on task/training data.
- Nonlinearities and down-sampling build increasingly hierarchical features.

Extra Readings

- [Torralba](#)
 - 22.1-2 , 4 Filter Banks
 - 23.1-2, 4 Image Pyramids
 - 28.1-2 Textures
 - 24.2-3, 7 CNNs
- [Szeliski](#)
 - 3.5 Pyramids
 - 10.5 Texture Analysis
 - 5.4 Convolutional Neural Networks
- [Klette](#)
 - 2.2.2 Regular Image Pyramids
 - 2.4.1 LoG, DoG and their Scale Spaces

FAQs

Filter Banks

Q: What is the relationship between a Gabor filter and a filter bank?

A: A Gabor filter is one kernel. A Gabor filter bank contains many Gabor kernels with different orientations, frequencies, scales, or phases. Together, they measure several kinds of local image structure.

Q: Why use Gabor filters instead of derivative filters?

A: If the goal is simply to estimate an image gradient, a derivative-of-Gaussian filter is more direct. Gabor filters are useful when we want a bank that distinguishes both orientation and spatial frequency, particularly for describing repeated patterns.

Q: What is the difference between odd and even Gabor filters?

A: The cosine component is symmetric about its centre and is called even. The sine component changes sign across its centre and is called odd. They represent two phase-shifted versions of the same oriented frequency pattern, i.e. $\sin(x + \pi/2) = \cos(x)$.

Q: Do odd and even filters detect different structures?

A: An odd filter responds strongly to an intensity transition, such as an edge. An even filter responds strongly to a centred bright or dark ridge, such as a line or bar. Their responses depend on the exact alignment and phase of the image pattern.

Q: Can a finite Gabor filter bank capture every possible pattern?

A: No finite bank can respond optimally to every image pattern. A practical bank samples a useful range of orientations, frequencies and scales. Its adequacy depends on the task and the structures expected in the data.

Q: What is the relationship between frequency and scale in a Gabor filter?

A: Frequency controls the spacing of the oscillations: higher frequency means narrower stripes, while lower frequency means wider stripes. Scale, usually controlled by the Gaussian σ , determines the size of the neighbourhood covered by the filter. They are related but not identical: frequency describes the pattern being measured, while scale describes how large an area is used to measure it.

Q: If frequency already determines pattern size, why do we also need a scale parameter?

A: Two Gabor filters can have the same stripe spacing but cover different-sized neighbourhoods. A larger Gaussian envelope observes more repetitions and uses more spatial context. In practice, frequency and scale are often changed together so that approximately the same number of cycles fits inside each filter.

Q: I read online that Gabor filters are complex-valued?

A: Yes. But for our purposes, it is sufficient to understand the even cosine filter and odd sine filter as a pair. In the complex formulation, cosine is the real component and sine the imaginary component.

Multi-Scale Reasoning

Q: Why smooth an image before downsampling it?

A: Downsampling removes spatial samples. Without smoothing, fine patterns can be misrepresented as false coarser patterns—an effect called aliasing. Gaussian smoothing removes details that the lower-resolution image can no longer represent reliably.

Q: Why does smoothing let us observe a larger scale?

A: Smoothing suppresses variations over small distances. The remaining image is dominated by structures that extend over larger neighbourhoods.

Q: Is applying one filter across a pyramid the same as changing the filter size?

A: They are two related ways to obtain scale coverage. At a higher pyramid level, each pixel represents a larger area of the original image, so the same pixel-sized filter has a larger effective receptive field.

Q: Is scale selection performed over an entire image region?

A: Usually, scale selection first compares $R(x, y; \sigma)$ across scales at a particular location. We may then search for maxima over both position and scale. Pooling over an entire region is a separate operation used for regional representations such as texture.

Q: What does the maximum response across scales tell us?

A: For a scale-normalized detector, the response tends to peak when the filter scale best matches the characteristic size of the local structure.

FAQs

Texture Representations

Q: Why pool filter responses when describing texture?

A: A response at one pixel only describes a highly localized pattern. Pooling statistics such as the mean, energy or variance over a region describes how patterns are distributed throughout that region and is less sensitive to their exact positions.

Q: Is the pooling window the same as a texture patch?

A: In this context, yes. The window or region Ω identifies the local patch whose responses are summarized into a texture vector.

Q: Does a texture representation imply segmentation?

A: No. One vector for an entire patch supports material classification. Computing localized vectors across an image can support segmentation. The same representation can be used differently depending on whether spatial location is retained.

Q: Are hand-designed texture representations still used?

A: Yes. Gabor filters and other classical texture descriptors remain useful when data is limited, computational cost or interpretability matters, or the patterns are well understood. Learned features are generally preferred for large, highly variable recognition problems. Current image-processing libraries still include Gabor-bank texture-classification workflows. [scikit-image example](#)

Q: What information is lost when filter responses are pooled?

A: Pooling discards the exact positions and arrangement of individual responses. It retains summary information about which patterns occur and how strongly they occur within the region. This trade-off makes the representation less sensitive to small translations.

Q: How should we choose the size of the pooling region?

A: The region should be large enough to contain representative repetitions of the texture, but small enough to avoid combining different materials or structures. When no single size is appropriate, statistics can be computed over several region sizes.

Q: Why might we use more than one statistic for each filter response?

A: Different statistics describe different properties. The mean measures the average signed response, while variance, energy or mean magnitude measures the overall strength of that pattern. Combining them usually describes the response distribution better than the mean alone.

Q: Is a texture vector automatically invariant to rotation?

A: No. An orientation-specific filter bank will generally produce different vectors when the texture rotates. Rotation invariance requires combining responses across orientations, aligning to a dominant orientation, or comparing against rotated examples.

Filter Banks to CNNs

Q: What exactly is learned in a convolutional layer?

A: The values of the convolution kernels are learned from training data. The number of layers, kernel sizes and other architectural choices are generally selected by the designer.

Q: Is max pooling learned?

A: Standard max pooling is not learned. Its rule, window size and stride are architectural choices. Some networks instead use learned strided convolutions for downsampling.

Q: Is the fully connected layer analogous to our earlier classifier?

A: Yes. Both map a feature representation to class scores. The key difference is that a CNN usually learns the feature representation and classifier jointly through the same training objective.

Q: Is a convolutional layer simply a learned filter bank?

A: Conceptually, yes. Each kernel produces one response map, and several kernels form a filter bank. The main difference is that CNN kernels are learned from task data rather than specified in advance.

Q: Why place a nonlinearity such as ReLU between convolutional layers?

A: Without nonlinearities, several convolutional layers would still represent one overall linear filtering operation. ReLU allows successive layers to combine responses conditionally and represent more complex patterns.

Q: Do deeper CNN layers still behave like ordinary image filters?

A: They still perform convolution, but they operate on earlier feature maps rather than directly on image intensities. Consequently, later layers respond to combinations of simpler patterns over progressively larger receptive fields.